

Plain versus Residual Convolutional Neural Networks

Implemented from Scratch in NumPy: A CIFAR-10 Depth Study

Srishti Pandey, Ahmed Abdelhamid
spandey@uni-osnabrueck.de, aabdelhamid@uos.de
Foundations of Machine Learning
Technical Report

Abstract—We implemented a convolutional neural network (CNN) in NumPy and used it to compare plain and residual architectures on CIFAR-10. The implementation includes custom convolution, ReLU, pooling, dense layers, dropout, softmax cross-entropy, SGD, and manual backward passes. The central question was whether skip connections help as depth increases. We tested one to four two-convolution blocks per stage. The residual block’s input-gradient backward pass was checked numerically in a projection-shortcut case, giving a median relative error of 7.34×10^{-13} . Residual models obtained slightly higher peak validation accuracy at two and three blocks per stage, but not at one or four blocks. The shallow plain CNN was the best configuration in the fixed-hyperparameter depth sweep. After learning-rate selection and a longer final training run, it reached a peak validation accuracy of 62.64% and a test accuracy of 61.56%. The results provide partial evidence that shortcut connections were associated with modestly higher peak validation accuracy at intermediate depths in this small normalization-free CNN. However, because the study used one run per condition and did not directly quantify gradient propagation or convergence speed, this pattern should be interpreted as consistent with, rather than conclusive evidence of, improved optimization.

I. PROBLEM DESCRIPTION

Adding layers gives a neural network more capacity, but it can also make training harder. A plain CNN passes information through every layer in sequence. In a residual block, the learned transformation $F(x)$ is combined with a shortcut $s(x)$:

$$y = F(x) + s(x). \quad (1)$$

When the input and output shapes match, $s(x) = x$. During backpropagation, this provides a direct path for the gradient in addition to the gradient through F . Residual learning was introduced to make substantially deeper networks easier to optimize [1].

This project tested whether the same idea helps in a smaller CNN built without automatic differentiation. We implemented every layer and its backward pass in NumPy. The question was: *does adding residual shortcuts change trainability and validation accuracy as the network becomes deeper, compared with a near-parameter-matched plain CNN?*

Rather than attempting to reproduce a full ResNet implementation or to achieve state-of-the-art CIFAR-10 performance, this study isolates the residual mechanism. It focuses on making the shortcut computation and its backward pass explicit, verifying the resulting gradients numerically, and evaluating the mechanism under controlled experimental conditions.

II. DATASET OVERVIEW

CIFAR-10 contains 60,000 colour images of size 32×32 in ten balanced object classes. The standard split is 50,000 training images and 10,000 test images [3]. We shuffled the standard training split and set aside 10,000 images for validation. The remaining 40,000 images were used for fitting. The official test set was used only for the final evaluation.

Images were reshaped to $(N, 3, 32, 32)$, converted to `float32`, and divided by 255 so that pixel values lay in $[0, 1]$. CIFAR-10 was a practical choice: it is large and varied enough to require non-trivial visual feature extraction, but its image size remains manageable for CPU-based NumPy convolution.

For this experiment, CIFAR-10 functions as a standardized computer-vision benchmark rather than as a model of biological or human visual processing. Its low-resolution, fixed-view images do not capture the contextual, active, multimodal, or few-shot conditions that characterize human object recognition. Its value in this report is therefore methodological: it provides a controlled image-classification task for evaluating how architectural choices affect optimization and generalization.

Table I
DATASET SPLIT.

Split	Images	Role
Training	40,000	Parameter updates
Validation	10,000	Architecture and learning-rate selection
Test	10,000	Final held-out evaluation

III. BASELINE NEURAL NETWORK

A. Implementation

The starting point was a plain CNN with one two-convolution block in each of three stages. Convolution was implemented with `im2col`: local image patches are rearranged into columns, so convolution becomes a matrix multiplication. The backward pass uses cached columns to calculate weight and bias gradients, then uses `col2im` to accumulate gradients back into the input layout.

The model used 3×3 kernels. Small kernels can learn local structure such as edges, colour boundaries, and textures; stacking them permits increasingly complex feature combinations. The three stages used 16, 32, and 64 channels. As spatial resolution decreases, more channels can be afforded without making computation prohibitively expensive. Stage transitions used stride-2 convolutions, reducing resolution from 32×32 to 16×16 and then to 8×8 .

The complete architecture was:

```
Conv(3→16) - ReLU - Stage 1(d) - Stage 2(d) - Stage
3(d) - Flatten - Dense(128) - ReLU - Dense(10).
```

The final dense layer maps learned features to ten class scores. The preceding 128-unit dense layer was chosen as a moderate-capacity classifier head, so that most of the model’s capacity remains in the convolutional feature extractor. `MaxPool2D` and `Dropout` were implemented in the notebook, but the reported models use strided convolution for downsampling and dropout was inactive ($p = 0$).

B. Training configuration

The loss function combined numerically stable softmax and cross-entropy. Weights used He initialization, $\sqrt{2/\text{fan_in}}$, which is designed for ReLU networks [2]. We trained with mini-batch SGD, batch size 64, and initial learning rate 0.01. Batch size 64 was a reasonable compromise between noisy updates and the memory demands of `im2col`. In the depth sweep, the learning rate was multiplied by 0.95 every five epochs. The baseline plain model ($d = 1$) had 600,106 trainable parameters and reached 60.58% peak validation accuracy over 30 epochs.

IV. ITERATIVE IMPROVEMENTS

A. Increasing depth

We increased the number of two-convolution blocks per stage from $d = 1$ to $d = 4$. This preserved the same overall stage structure while increasing the number of nonlinear transformations. The deeper plain networks could fit the training data, but they did not improve validation performance over the shallow baseline.

B. Adding residual shortcuts

A plain block was defined as

$$\text{PlainBlock}(x) = \text{ReLU}(\text{Conv}_2(\text{ReLU}(\text{Conv}_1(x)))) \quad (2)$$

The residual block retained the same main branch and added a shortcut:

$$\text{ResidualBlock}(x) = \text{ReLU}\left(\frac{F(x) + s(x)}{\sqrt{2}}\right) \quad (3)$$

The shortcut was the identity when dimensions matched. At stage transitions, a learned 1×1 projection convolution matched channel number and spatial resolution.

The backward pass split the incoming gradient after the final ReLU. One copy was propagated through the two convolutions; the other was propagated directly through the identity shortcut or through the projection convolution. The two resulting input gradients were added. A finite-difference gradient check of a projection-shortcut block gave a median relative error of 7.34×10^{-13} , well below the acceptance threshold of 10^{-6} . Gradient checks were also passed for the identity-shortcut case (no dimension change) and the strided-projection case (spatial downsampling with channel change), confirming correct backward-pass behavior across all three shortcut configurations used in the depth sweep.

C. Stabilizing the residual sum

The first residual implementation used $F(x) + s(x)$ without scaling. Because two similarly scaled signals are added, activation magnitude grew across blocks. We observed approximately tenfold growth in activation standard deviation by block six, followed by numerical overflow and NaN losses at moderate depth. We therefore divided the sum by $\sqrt{2}$. This reduced activation growth and made the normalization-free residual models trainable.

D. Controlling capacity and tuning learning rate

Residual models required two 1×1 projection shortcuts. They had 2,656 more parameters than the plain models at every depth, so the comparison is near-parameter-matched rather than exactly matched. After the depth sweep, we tested five initial learning rates for the strongest configuration.

Table II
PARAMETER COUNTS FOR ALL TESTED DEPTHS.

d	Plain	Residual	Difference	Increase
1	600,106	602,762	2,656	0.44%
2	697,098	699,754	2,656	0.38%
3	794,090	796,746	2,656	0.33%
4	891,082	893,738	2,656	0.30%

V. EVALUATION

We used peak validation accuracy as the main metric because validation loss increased after its minimum in several runs. This meant that final-epoch accuracy did not always reflect the best generalizing model.

For context, modern CIFAR-10 architectures with batch normalization, momentum-based optimizers, and data augmentation typically exceed 90% test accuracy; the substantially lower accuracies reported here reflect the deliberately constrained scope of this study; an unaugmented, unnormalized,

from-scratch implementation intended to isolate the residual mechanism rather than to maximize benchmark performance.

Table III
PEAK VALIDATION ACCURACY BY ARCHITECTURE AND DEPTH.

Blocks/stage	Plain	Residual	Residual – Plain
1	60.58%	58.67%	−1.91 pp
2	57.19%	58.55%	+1.36 pp
3	56.57%	57.66%	+1.09 pp
4	57.28%	56.27%	−1.01 pp

Residual shortcuts were associated with a small peak-validation-accuracy advantage at two and three blocks per stage: +1.36 percentage points at depth two and +1.09 percentage points at depth three. However, the pattern was not monotonic: the shallow plain CNN was the best model in the fixed-hyperparameter depth sweep, and the residual model did not outperform the plain model at depth four. Thus, under this specific training configuration, residual shortcuts were associated with better intermediate-depth validation performance rather than a general guarantee that they prevent performance degradation. Figure 1 summarizes the final and peak validation accuracies across the tested configurations.

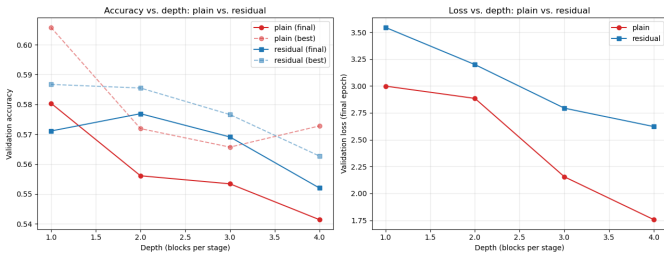


Figure 1. Final-epoch and peak validation accuracy of the plain and residual CNNs across network depths.

The loss curves show that all conditions substantially reduced training loss. Validation loss later increased in several cases while training loss continued decreasing, which indicates overfitting. The final selected model reached 94.07% training accuracy but a highest validation accuracy of 62.64%. Figure 2 presents the corresponding training and validation loss trajectories.

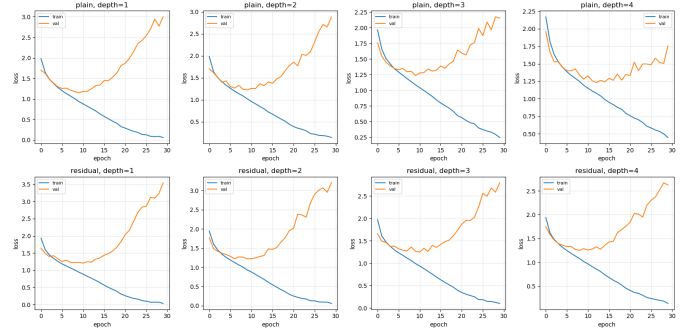


Figure 2. Training and validation cross-entropy loss trajectories for each architecture-depth condition.

To select the final training configuration, we evaluated five initial learning rates for the selected plain CNN with one block per stage. The resulting validation accuracies are reported in Table IV.

Table IV
LEARNING-RATE SWEEP FOR THE SELECTED PLAIN CNN WITH ONE BLOCK PER STAGE.

Initial learning rate	Best validation accuracy
0.001	50.88%
0.003	57.64%
0.010	60.34%
0.030	57.81%
0.100	59.84%

The learning-rate sweep selected 0.01. The selected plain depth-one configuration was retrained for up to 60 epochs with learning-rate decay and validation-loss early stopping. It reached its highest validation accuracy of 62.64% at epoch 12; early stopping was triggered at epoch 22. The last retained model achieved 61.56% test accuracy and a test loss of 2.0648. Because the best validation-loss checkpoint was not restored before testing, this test score does not necessarily correspond to the best-validation model. Figure 3 shows the class-level prediction pattern on the test set. The confusion matrix shows that misclassifications concentrate among visually similar animal classes: cat was confused with dog 256 times, far more than with any other class, while dog was confused with cat 88 times. This asymmetric cat/dog confusion is a well-documented difficulty in CIFAR-10 classification, reflecting shared low-level visual features (fur texture, similar poses, similar backgrounds) rather than a specific weakness of the residual mechanism. Vehicle classes (automobile, ship, truck) show comparatively few cross-confusions, suggesting the model separates coarse object categories more reliably than fine-grained animal distinctions.

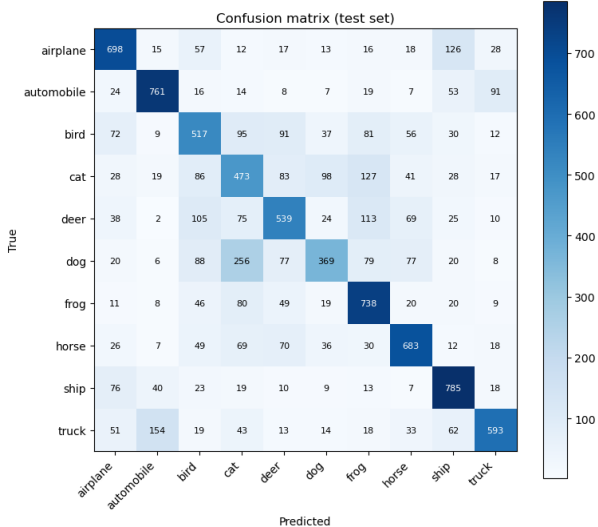


Figure 3. Confusion matrix of test-set predictions produced by the final selected plain CNN.

VI. LEARNINGS AND CHALLENGES

The main implementation lesson was that residual learning is not simply a forward addition. It requires correct gradient routing through two paths, shape matching when dimensions change, and numerical verification. The gradient check was necessary before drawing conclusions from the training curves.

The results also separate representational capacity from learnability. A deeper architecture can in principle represent more complex functions, but may not generalize better under finite optimization resources. More generally, the experiment demonstrates that network topology influences the efficiency with which a hierarchical system propagates information and gradients during learning. Although residual shortcuts are not intended as a direct model of biological neural pathways, they provide a computational example of how pathway structure can constrain learning dynamics.

A practical limitation distinct from the optimization findings above was computational cost: because the implementation relies on NumPy rather than a GPU-accelerated framework, per-epoch training time scaled substantially with depth, from roughly 190 seconds per epoch at one block per stage to over 900 seconds at four blocks per stage in the residual condition. This CPU-bound scaling constrained the feasible range of depths and epochs explored, and would need to be addressed (e.g., via GPU acceleration or a compiled backend) to extend this study to the deeper regimes where residual connections are expected to matter more.

The results should be interpreted carefully. Overfitting was substantial, and the training regime intentionally omitted augmentation, normalization, weight decay, momentum, and adaptive optimization. Each architecture-depth condition was evaluated only once, and minibatch shuffling was not controlled by a fixed seed. The observed intermediate-depth residual advantage should therefore be interpreted as descriptive rather than as a robust estimate of a general architectural effect.

In addition, the learning-rate sweep used up to 20 epochs without learning-rate decay, whereas the depth sweep and final training run used scheduled decay. The sweep therefore served as a practical learning-rate selection step rather than a strictly protocol-matched comparison.

Finally, the final training call did not explicitly restore the best validation-loss checkpoint before test evaluation. The reported test score therefore reflects the last retained early-stopped model rather than necessarily the model with the minimum validation loss. Future runs should save the best weights, restore them after training, and then evaluate the restored model once on the test set.

VII. CONCLUSION

We implemented a CNN from scratch in NumPy and compared plain and residual blocks on CIFAR-10. The residual backward pass was verified numerically, and residual-sum scaling was needed to avoid activation growth without batch normalization. Residual models showed modest peak-validation-accuracy advantages at intermediate depths, but not at every tested depth. The shallow plain CNN was the best model in the fixed-hyperparameter depth sweep; after learning-rate selection and longer retraining, its last retained early-stopped model achieved 61.56% test accuracy. The project shows that residual shortcuts can be associated with improved performance in some deeper configurations, but that this result also depends on activation stability, regularization, optimization choices, and reliable repeated evaluation across random seeds.

REFERENCES

- [1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proceedings of the IEEE International Conference on Computer Vision*, 2015, pp. 1026–1034, doi: 10.1109/ICCV.2015.123.
- [3] A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images*. University of Toronto, 2009. [Online]. Available: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>